



PROCEDIMIENTOS Y FUNCIONES BASE DE DATOS EMPRESA

1. Crea un procedimiento de nombre `listEmpleadosConHijos` que muestre los empleados que tienen hijos.
2. Crea un procedimiento de nombre `contarEmpleados` que muestre la cantidad de empleados.
3. Crea un procedimiento de nombre `updSalarioEmpleados` que incremente el salario de los empleados un 10%.
4. Crea un procedimiento de nombre `listDepartamentos` que liste los departamentos de un determinado centro. Habrá que pasar como parámetro el nombre del centro
5. Crea un procedimiento de nombre `listDepartamentosPlus` que liste los departamentos de un centro, y en el caso de recibir como argumento un valor nulo, devuelva todos los departamentos.
6. Crea un procedimiento de nombre `updSalarioEmpleadosParam` que incremente el salario de los empleados una determinada cantidad a partir de un parámetro de entrada.
7. Crea un procedimiento de nombre `contarEmpleados` que devuelva la cantidad de empleados en un parámetro de salida.
8. Crea un procedimiento de nombre `contarEmpleadosDpto` que devuelva la cantidad de empleados de un determinado departamento (introducido vía un parámetro de entrada) en un parámetro de salida.
9. Crea un procedimiento de nombre `sueldosSet` que devuelva el sueldo menor, el mayor y el promedio de todos los empleados (usando SET).
10. Crea un procedimiento de nombre `sueldosSelectInto` que devuelva el sueldo menor, el mayor y el promedio de todos los empleados (usando una única sentencia SELECT INTO).
11. Crea un procedimiento de nombre `semanalf` que reciba como entrada un entero que represente un día de la semana y que devuelva una cadena con el nombre del día de la semana correspondiente (utilizando IF o CASE). Por ejemplo, para la entrada 1 debería devolver Lunes.
12. Crea un procedimiento de nombre `5insertaHabilidad` que reciba como entrada un código de habilidad y su descripción, y que sólo la inserte si el código de la habilidad tiene un tamaño de 5 caracteres.
13. Crea un procedimiento de nombre `upsertHabilidad` que reciba como entrada un código de habilidad y su descripción, y que sólo la inserte si el código de la habilidad tiene un tamaño de 5 caracteres. Si el código ya existe, debe modificar la habilidad con la nueva descripción, y si no, la insertará.



14. Crea las funciones `contarEmpleados` y `contarEmpleadosDpto` reescribiendo los procedimientos `contarEmpleados` y `contarEmpleadosDpto` de las actividades anteriores.
15. Crea la función `presupuestoCentro` que, a partir del código de un centro, devuelva su presupuesto (calculado como la suma de los presupuestos de sus departamentos).
16. Crea la función `totalHabilidadesEmpleado` que, a partir de un código de un empleado, devuelva cuantas habilidades tiene.
17. Crea la función `directorCentro` que, a partir del código de un centro, devuelva el nombre de su director.
18. Crea la función `emailEmpleado` que, a partir de un código de empleado, devuelva su email con la siguiente nomenclatura: `CodEmp@CodDep.CodCen.com`
19. Crea la función `validaHijosEmpleados` que a partir de un código de un empleado, compruebe si la cantidad de hijos de la tabla `empleado` coinciden con los de la tabla `hijo`.
20. En la base de datos empresa, se pide crear un conjunto de procedimientos que colaboren entre sí para generar un informe salarial por centro de trabajo. Para ello:
21. Crea el procedimiento `mediaSalarial` que, a partir de un código de departamento (parámetro de entrada), devuelva en un parámetro de salida el salario medio de los empleados de dicho departamento.
22. Crea la función `categoriaDepartamento` que, a partir de un código de departamento (parámetro de entrada), devuelva la categoría según el salario medio de sus empleados:
 - bajo si el salario medio es inferior a 2.000.000
 - medio si está entre 2.000.000 y 5.000.000
 - alto si es superior a 5.000.000

Esta función debe invocar al procedimiento `mediaSalarial` para obtener la media.

Haciendo uso de cursores y la base de datos empresa, se pide:

23. Crea la tabla `empleado_copia` con la misma estructura que la tabla `empleado` (mediante `CREATE TABLE .... LIKE ...`).
24. Crea el procedimiento `empleadosSinHijos`, para que, utilizando cursores y recuperando todos los empleados que no tienen hijos en la consulta, recorra el resultado e inserte en `empleado_copia` aquellos empleados que no tiene hijos.
25. Crea el procedimiento `empleadosNumHijos`, para que, utilizando cursores con parámetros, recorra la tabla de empleados e inserte en `empleado_copia` aquellos empleados que tienen la cantidad de hijos recibidos como un parámetro de entrada del procedimiento.



26. Crea la tabla empleado_plus con los mismos datos que la tabla empleado (mediante CREATE TABLE AS SELECT ...)
27. Sobre la nueva tabla empleado_plus, añade la columna HijMayor que permita almacenar el nombre del hijo mayor de cada empleado. Para ello, crea el procedimiento almacenado procHijos, y mediante un cursor, sobre los hijos mayores de cada empleado, modifique el campo del empleado y almacene su nombre.
28. Crea una función llamada funcionHabilidadesEmpleado que a partir del código de un empleado, y utilizando un cursor, devuelva las habilidades de dicho empleado separadas por comas. Es decir, para el empleado 1 deberá almacenar GEREN,RELPU. No puedes utilizar la función GROUP_CONCAT.
29. Sobre la nueva tabla empleado_plus, añade la columna DescHabs que permita almacenar los códigos de las habilidades. A continuación, crea el procedimiento almacenado procHabilidadesEmpleados que complete dicha columna utilizando la función funcionHabilidadesEmpleado.